

Week 3 Exercises. (partial) Solutions for the exercises slide.

Introduction: Fibonacci Numbers

1. Yes, we can! We only really need 3 integer variables.

```
function F(int n) \\assumes n >= 2
  memoFminus2 := 0; memoFminus1 := 1;
  for int j from 2 ... n
    memoF := memoFminus1 + memoFminus2;
    memoFminus2 := memoFminus1;
    memoFminus1 := memoF;
  return memoF
end function
```

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

1. Both books give implementation, see [GT 12.5] and [CLRS 15.4].
2. To store an LCS of length t of two strings (one of length n and the other of length m , $n \leq m$) involves storing two sequences of t indices (one for each string). Notice that $t \leq \min\{n, m\} = n$.
 - (i) Naively updating these entries means we need to copy such a pair sequence each time we process a table entry. This increases the time from $O(1)$ to $O(n)$. This means $O(n^2m)$ time in total to compute the table.
 - (ii) For entry (i, j) , we store two length $L(i, j) \leq n$ sequences, ie, each table entry would require $O(n)$ units of additional space. Thus, the table would have size $O(n^2m)$. However, we can be a bit clever. Since we do not need to backtrack, we can instead just maintain the previous column together with the current column that is being computed. In this way, the space needed reduces to $O(n^2)$.

Scheduling Telescope Time [GT 12.3]

1. This is also given in GT 12.3, but we include it here using the notation from the slides.

```
# requires A sorted by finish times.
function BottomUpOpt(array A, array p):
  create array OPT; set OPT[0] := 0;
  For each i from 1 to n
    OPT[i] = max( OPT[i-1], A[i].b + OPT[ p[i] ] )
  return OPT[n]
end function
```

Runtime: The loop involves n iterations. Each iteration just involves the comparison of two numbers, i.e., can be performed in $O(1)$ time. Thus, $O(n)$ in total.

2. To compute the predecessor array p , proceed as follows. Note that we already have A sorted by finish times.

For each i from 1 to n

- binary search A to find index j such that $A[j].f \leq A[i].s$ and $A[j+1].f > A[i].s$;
- set $p[i] = j$

Runtime: There are n iterations in total. Each iteration is just a binary search on a sorted list and this can be performed in $O(\log n)$ time. Thus, $O(\log n)$ time per iteration, and $O(n \log n)$ time in total to compute the array p .

Knapsack [GT 12.6], [CLRS 16.2 (+exercise 35-7)]

1. Coin Changing

- (a) We can translate this problem into an instance of Knapsack with repetitions, and just run the algorithm to solve Knapsack. Set the weight $W := v$, and create n items where item i has weight and value $d[i]$. Now, it is possible to make change for value v if and only if the optimal value that fits into the Knapsack with weight limit W is also W . The running time is $O(nW) = O(nv)$.
- (b) Here we can use the same approach as in (a) but now on Knapsack without repetition.
- (c) Instead of storing the value in the table, we store the number of coins used to reach that value, or No if the entry (i, W) has no solution.

Travelling Salesperson (TSP)

1. (omitted) The idea is very similar to the Knapsack backtracking.

2. (i) For the entry (S, v) , store a cheapest path from s to v using all the vertices of S .

(ii) Similar to the LCS problem, we need to copy the path we are extending. If we are in iteration j , this makes for $O(j)$ extra work per entry.

(iii) For each j , we need to use $(j-1) \cdot \binom{n}{j-1}$ entries to construct the next $j \cdot \binom{n}{j}$ entries. So, in our original table we had $\sum_{j=1}^n j \binom{n}{j} = n2^{n-1}$ entries. In the revised table, we only keep the entries needed for sets of size $j-1$ to populate the new part of the table for sets of size j . This is maximized when $j = \lceil \frac{n}{2} \rceil$, for which we need to store $\leq n \cdot \binom{n}{\lceil \frac{n}{2} \rceil}$ entries.

A nice savings of memory :)

Going a bit further, $\binom{n}{\frac{n}{2}}$ is at most $2^n / \sqrt{n}$ (via Sterling's approximation). Comparing this to 2^n to the binomial term $\binom{n}{\frac{n}{2}}$, the latter is roughly $\sqrt{n}$ smaller.